



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Vicerrectorado Académico

Proyecto de carrera: Ingeniería en Informática

Asignatura: Ingeniería del Software II

Unidad 6: Liderazgo, Equipos y Cultura de Ingeniería

Integrantes	C.i
Arnaldo Perdomo	30.791.551
Daniel Reyna	29.989.066
Jhoanny Maita	30.694.732
Yoryelis Ocando	31.074.651

Puerto Ordaz, 23 de Septiembre del 2026

Unidad 6:Liderazgo, Equipos y Cultura de Ingeniería

1. Pregunta:

La Maniobra Inversa de Conway y Team Topologies Contexto, una empresa bancaria tiene una arquitectura de software monolítica. Su estructura organizacional es funcional: un "Departamento de Base de Datos", un "Departamento de Backend Java" y un "Departamento de UI". Cada vez que quieren lanzar una nueva funcionalidad de "Créditos", los tres departamentos deben coordinarse mediante un Gerente de Proyecto tradicional, generando demoras de meses y constantes conflictos. Analice el problema actual utilizando la Ley de Conway. ¿Por qué la estructura organizacional está garantizando el fracaso de la agilidad?

Proponga un rediseño organizacional utilizando los 4 tipos de equipos de Team Topologies (Stream-aligned, Platform, Enabling, Complicated-subsystem).

¿Cómo quedarían conformados los equipos para soportar una futura migración a microservicios?

¿Qué rol jugaría el antiguo "Gerente de Proyecto" en esta nueva estructura organizacional?

Diagnóstico usando la Ley de Conway

Lo primero que hay que entender en este caso es que el problema del banco no es solo técnico ni de metodologías. La Ley de Conway dice, básicamente, que las organizaciones están condicionadas a producir diseños que son copias de sus estructuras de comunicación. O sea, si la empresa está dividida en departamentos que casi no se hablan entre sí, el software que construyan va a reflejar esas mismas divisiones. No es magia, es sentido común: la forma en que la gente se comunica termina definiendo cómo se diseña el sistema.

En el banco tenemos un Departamento de Base de Datos, un Departamento de Backend Java y un Departamento de UI. Cada uno vive en su mundo, con sus prioridades, sus jefes y sus tiempos. Cuando quieren lanzar una funcionalidad de "Créditos", los tres departamentos tienen que coordinarse entre sí. Como no existe un equipo que sea dueño del crédito de punta a punta, todo pasa por un Gerente de Proyecto tradicional que hace de intermediario. Ese gerente no tiene autoridad real sobre la gente, porque en una estructura funcional los recursos se reportan a los gerentes de área. Entonces empieza la cadena de reuniones, correos, tickets y favores para que los tres departamentos se pongan de acuerdo. Ahí ya tenemos meses de demora asegurados.

El problema de fondo es que la estructura está armada por capas tecnológicas, no por flujo de valor. Nadie es responsable del resultado final en producción. Cada departamento optimiza lo suyo: Base de Datos quiere estabilidad y normalización, Backend quiere lógica y servicios, UI quiere usabilidad. Cuando algo falla, todos se echan la culpa. Y cuando hay que cambiar algo, por pequeño que sea, hay que coordinar los tres silos. Eso genera handoffs constantes, o sea, el trabajo pasa de un departamento a otro como si fuera una carrera de relevos, pero en cada pase se pierde contexto, se reinterpretan requisitos y aparecen dependencias que nadie había

previsto. Lo que en un equipo cross-functional sería una conversación de una tarde, acá se convierte en semanas de ida y vuelta.

¿Por qué esto garantiza el fracaso de la agilidad? Porque la agilidad necesita equipos autónomos, con todas las capacidades, que puedan decidir y entregar sin depender de otros. Acá no existe un equipo que sea dueño de “Créditos” de principio a fin. Existen tres equipos que son dueños de capas. Entonces cualquier intento de ser ágil choca contra la pared: no puedes hacer sprints si para cada historia necesitas que tres departamentos se coordinen. No puedes tener feedback rápido si el ciclo de entrega depende de colas y aprobaciones cruzadas. No puedes tener autonomía si el Gerente de Proyecto tiene que rogar por prioridad en cada área.

Y esto se refuerza sólo: como la comunicación es por capas, el monolito es por capas. Y como el monolito es por capas, cualquier cambio obliga a tocar los tres departamentos. Es un círculo vicioso. La Ley de Conway se cumple y además se retroalimenta. Por eso se puede decir que la estructura organizacional está garantizando el fracaso de la agilidad. No es que los desarrolladores sean malos ni que el PM sea incompetente; es que la organización los obliga a moverse lento. Mientras no se cambie la estructura de comunicación y no se creen equipos alineados al flujo de valor, cualquier framework ágil que se implemente va a terminar ahogado en reuniones, handoffs y conflictos entre departamentos. El diagnóstico es claro: el problema no está en el código, está en cómo está pensada la empresa.

Rediseño con Team Topologies

Si el diagnóstico con la Ley de Conway nos dice que la estructura funcional del banco (Base de Datos, Backend Java, UI) está condenada a producir un monolito acoplado, entonces la solución no puede ser solo técnica. Hay que rediseñar la organización a propósito, para que la comunicación entre las personas empuje hacia microservicios desacoplados en vez de hacia un monolito por capas. A esto se le llama la maniobra inversa de Conway: en lugar de dejar que la estructura organizacional imponga la arquitectura por accidente, se diseña la estructura deliberadamente para que produzca la arquitectura que queremos.

Para eso usamos el modelo de Team Topologies, que propone cuatro tipos de equipo, cada uno con una función distinta:

- El primero es el equipo de flujo (stream-aligned team). En el caso del banco, sería el Equipo de Créditos: un equipo multifuncional, con gente de backend, frontend y suficiente conocimiento de base de datos como para operar su propio microservicio sin depender de otro departamento para cada cambio. Este equipo es dueño del crédito de punta a punta: diseño, desarrollo, pruebas y despliegue. Ya no hay que coordinar tres silos para lanzar algo, porque el equipo entero tiene todo lo que necesita adentro.

- El segundo es el equipo de plataforma (platform team). Sería la Plataforma de Ingeniería, que en la práctica absorbe buena parte de lo que hoy hace el Departamento de Base de Datos, pero ya no como intermediario burocrático sino como un servicio autoservible: CI/CD, provisión de bases de datos, observabilidad, contenedores. La idea es que el Equipo de Créditos no tenga que aprender a levantar un clúster ni pedir permiso para una base de datos nueva; simplemente consume la plataforma como si fuera un producto.
- El tercero es el equipo habilitador (enabling team). Sería un pequeño grupo de arquitectos o seniors que ayuda temporalmente a los equipos de flujo a adquirir capacidades que les faltan, por ejemplo, a migrar su módulo del monolito hacia un microservicio independiente. Lo importante acá es que no hace el trabajo por ellos ni se convierte en un cuello de botella permanente: su meta es transferir el conocimiento y dejar de ser necesario en ese equipo.
- El cuarto, que solo aplica si existe una pieza realmente compleja, es el equipo de subsistema complicado (complicated-subsystem team). Si el banco tiene, por ejemplo, un motor de cálculo de score crediticio con lógica actuarial pesada, ese motor se aísla en su propio equipo especializado. Así los equipos de flujo lo consumen como un servicio, sin tener que entender sus internos.

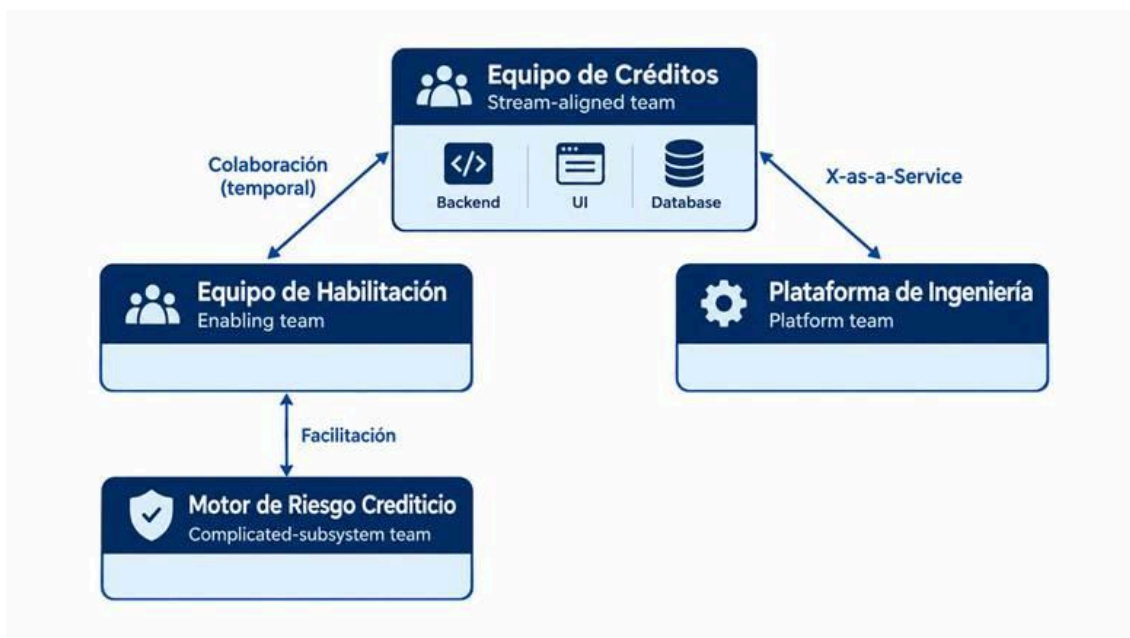
Con esto, la estructura quedaría más o menos así: el Departamento de Backend Java se convierte en el Equipo de Créditos (y en el futuro, en equipos equivalentes para Cuentas, Pagos, etc., cada uno dueño de su dominio). El Departamento de Base de Datos se convierte en la Plataforma de Ingeniería. El Departamento de UI deja de existir como silo aparte y sus integrantes se reparten dentro de cada equipo de flujo, para que el frontend deje de ser un handoff y pase a ser parte del mismo equipo que entrega el microservicio. Los arquitectos o seniors forman el equipo habilitador que acompaña la migración del monolito. Y si hace falta, se separa un equipo de subsistema complicado para la lógica de riesgo.

En cuanto a cómo interactúan estos equipos entre sí, Team Topologies define tres modos. La colaboración se da entre el Equipo de Créditos y el equipo habilitador mientras dura la migración activa del monolito: trabajan codo a codo, con comunicación constante, mientras se transfiere el conocimiento. El modo X-as-a-Service se da entre el Equipo de Créditos y la Plataforma de Ingeniería: el equipo de Créditos consume infraestructura, pipelines y bases de datos como un servicio, sin necesidad de coordinación humana constante ni de abrir tickets a otro departamento. Y la facilitación es lo que hace el equipo habilitador de forma más amplia: ayuda puntual y temporal a cualquier equipo

de flujo que necesite adoptar una práctica nueva, como testing automatizado o diseño de APIs, sin quedarse metido ahí de forma permanente.

Queda pendiente qué pasa con el antiguo Gerente de Proyecto. Mi propuesta es que se transforme principalmente en Engineering Manager del equipo de flujo, con un poco de Agile Coach mientras dura la transición. La razón es que el Gerente de Proyecto tradicional existía justamente porque la estructura funcional necesitaba a alguien que coordinara a mano los handoffs entre departamentos. Si eliminamos los silos, esa función de "coordinador de entregas" deja de tener sentido, porque el equipo ya es dueño de punta a punta de su propio flujo de valor. Como Engineering Manager, se enfocaría en el desarrollo de las personas del equipo, en la carga de trabajo y en la salud general del equipo, no en perseguir tareas entre departamentos. Y mientras la organización todavía está aprendiendo a funcionar así, puede actuar también como Agile Coach o facilitador, ayudando a que los nuevos modos de interacción funcionen y a que la plataforma o el equipo habilitador no terminen convirtiéndose en un nuevo cuello de botella disfrazado. No creo que la evolución natural sea a Product Owner, porque ese rol necesita visión de producto y priorización de negocio, que es un perfil distinto al de coordinación que tenía el PM; podría ser una función que cubra otra persona del equipo, pero no es lo que este puesto se convierte de forma natural.

Diagrama "Después"



2. Pregunta:

Liderazgo, Conflicto y Seguridad Psicológica Contexto: Usted es el nuevo Agile Coach / Líder Técnico de un equipo. Nota que en las revisiones de código (Pull Requests), el Arquitecto Principal (un desarrollador muy senior y brillante) utiliza comentarios sarcásticos y destructivos hacia los desarrolladores junior, tales como ¿En serio vas a usar este patrón? Es de principiantes. Los juniors han dejado de proponer ideas en las retrospectivas y la velocidad de entrega ha caído.

Diagnostique la situación utilizando el concepto de Seguridad Psicológica de Amy Edmondson. ¿Qué impacto tiene esto en el rendimiento del equipo a largo plazo?

Diseñe una estrategia de intervención utilizando principios de Liderazgo de Servicio y Comunicación No Violenta. ¿Cómo abordaría el conflicto con el Arquitecto Principal en privado y cómo reestructuraría el proceso de revisión de código para objetivar el feedback?

Proponga una dinámica para la próxima Retrospectiva que ayude a reconstruir la confianza del equipo.

Diagnóstico con Seguridad Psicológica

Edmondson (1999) define la seguridad psicológica como la creencia compartida por los miembros de un equipo de que este es un entorno seguro para asumir riesgos interpersonales: preguntar, admitir errores, pedir ayuda, proponer ideas poco habituales o señalar un problema sin temor a ser humillado, ignorado o penalizado por ello. No se trata de un rasgo individual de personalidad, sino de una propiedad emergente del clima del equipo, construida o destruida por las interacciones cotidianas entre sus miembros, especialmente por las de quienes tienen mayor estatus o poder técnico.

El propio proyecto Aristóteles de Google citado en el material de la unidad identificó la seguridad psicológica como el predictor número uno del alto rendimiento en equipos de ingeniería de software por encima de variables como la seniority individual promedio o la composición del equipo. Esto la convierte en la condición sine qua non para que existan retrospectivas honestas, innovación real y aprendizaje colectivo.

¿Cómo el miedo al ridículo bloquea el aprendizaje, la innovación y la velocidad?

En el caso planteado el comportamiento del Arquitecto Principal introduce lo que Edmondson llama un impuesto al riesgo interpersonal, cada vez que un junior propone algo y recibe una respuesta sarcástica, el costo social de participar aumenta y el equipo aprende de forma implícita que es más seguro callar que exponerse, por lo que este mecanismo afecta tres dimensiones críticas:

- **Aprendizaje:** el desarrollo de software es una actividad de conocimiento donde el error y la pregunta son el principal vehículo de aprendizaje. Si preguntar algo redundante o proponer un patrón distinto se castiga con ridículo, los junior optan por el silencio antes que por el aprendizaje activo así deteniendo su curva de crecimiento en lo técnico.

- Innovación: toda idea nueva es por definición una desviación de lo establecido y por tanto un riesgo. Así que si el Arquitecto castiga la desviación en lugar de discutirla técnicamente, el equipo converge hacia una única fuente de decisión es decir el propio Arquitecto, perdiendo la diversidad de soluciones que en Team Topologies y en la cultura DevOps se busca precisamente fomentar mediante la autonomía de equipos multifuncionales.
- Velocidad de entrega: al perder la voz de los junior en las sesiones de retroalimentación y Pull Requests, el equipo deja de detectar cuellos de botella, impedimentos y errores tempranos, lo cual genera lo contrario es decir que ralentiza el flujo de trabajo. La disminución de velocidad que hemos notado en este caso es de hecho un claro indicativo de una falta de seguridad psicológica, y que no tiene que ver con la capacidad técnica del equipo.

Impacto a largo plazo en rendimiento, retención y calidad

Si la dinámica actual no se corrige sus efectos dejan de ser puntuales y se convierten en estructurales. Y a continuación la siguiente matriz resume el impacto observado a corto plazo frente a su proyección a largo plazo en las cuatro dimensiones más afectadas por la pérdida de seguridad psicológica en este equipo.

Dimensión	Efecto a corto plazo	Efecto a largo plazo
Rendimiento del equipo	Caída en la velocidad de entrega throughput, los juniors dejan de participar en retrospectivas.	El estancamiento técnico del equipo es menos experimentación, menos mejora continua y dependencia excesiva del Arquitecto como único punto de decisión.
Retención del talento	Desmotivación y desconexión emocional de los desarrolladores junior.	Rotación turnover, de personal junior, pérdida de conocimiento tácito y alto costo de reclutamiento y onboarding para reemplazarlos.
Calidad del software	Los Pull Requests se aprueban sin discusión real, los juniors evitan señalar errores del Arquitecto.	Aumento de deuda técnica y de defectos no detectados a tiempo, al perderse la revisión crítica cruzada que exige un code review saludable.
Innovación y aprendizaje	Los juniors dejan de proponer ideas o patrones alternativos por miedo al ridículo.	El equipo pierde diversidad de pensamiento, se reduce la capacidad de adaptación ante nuevos retos, menor maestría y autonomía percibidas, Pink, 2009.

En conjunto estos efectos configuran un ciclo de deterioro, a menor seguridad psicológica menor participación, a menor participación, menor calidad y menor velocidad, y ante ese menor desempeño visible, el riesgo natural es que la organización responda con más control y supervisión lo cual reduce aún más la autonomía y profundiza el problema en lugar de resolverlo.

Liderazgo de Servicio

El Líder Técnico no debe actuar como un jefe que impone una sanción, sino como un servidor que elimina el obstáculo que está frenando al equipo. Bajo este enfoque, el objetivo no es castigar al Arquitecto Principal, sino restaurar las condiciones para que el equipo vuelva a proponer ideas y a aprender sin miedo. El líder asume la responsabilidad de intervenir directamente, en lugar de esperar a que los juniors se defiendan solos, porque proteger al equipo de interferencias externas o internas es parte del rol de servicio.

Comunicación No Violenta

Para abordar el conflicto sin generar un enfrentamiento defensivo, conviene estructurar la conversación en cuatro pasos:

1. **Observación sin juicio:** describir lo que ocurrió como un hecho, sin calificarlo. Por ejemplo, señalar que en las últimas revisiones de código hubo comentarios como "es de principiantes", sin decir todavía que eso está mal.
2. **Sentimiento:** expresar el efecto que esto genera en el equipo, evitando culpar directamente. Por ejemplo, notar que el equipo se ve más callado en las retrospectivas y que la participación bajó.
3. **Necesidad:** explicar qué necesita el equipo para funcionar bien, que es un ambiente donde las personas puedan proponer ideas y equivocarse sin ser humilladas.
4. **Petición concreta:** pedir un cambio de comportamiento específico y verificable, no una actitud general. Por ejemplo, pedir que el feedback técnico se enfoque en el código y no en la persona, y que se formule como pregunta o sugerencia en vez de burla.

Abordaje privado con el Arquitecto Principal

La conversación debe ser en privado, nunca frente al equipo, para no generar una humillación pública que reproduzca el mismo daño que se busca corregir. Es importante reconocer primero el valor técnico del Arquitecto, ya que su experiencia es real y necesaria, y dejar claro que el problema no es su nivel de exigencia sino la forma en que lo comunica. El mensaje central es que la exigencia técnica y el respeto no son opuestos, y que un comentario sarcástico no mejora la calidad del código, solo cierra la puerta a que otros participen. Conviene cerrar la conversación acordando un cambio de comportamiento concreto, y no solo una intención vaga de "mejorar".

Rediseño del proceso de revisión de código

El objetivo es que el feedback se vuelva objetivo y esté basado en criterios compartidos, no en la opinión personal de quien revisa. Algunas medidas:

- Adoptar una guía de estilo y un checklist de revisión escritos, para que las observaciones se apoyen en un estándar acordado por el equipo y no en el gusto individual del revisor.
- Establecer que todo comentario debe referirse al código, nunca a la persona, y debe venir acompañado de una razón técnica o una alternativa concreta, no sólo una crítica.
- Rotar quién revisa cada Pull Request, para que la responsabilidad de dar feedback no recaiga siempre en la misma persona y los juniors también practiquen ese rol.
- Usar etiquetas de severidad en los comentarios, por ejemplo distinguir entre una observación obligatoria y una sugerencia opcional, para que el tono quede fuera de la evaluación técnica.

Dinámica para la Retrospectiva

Para reconstruir la confianza sin señalar directamente al Arquitecto ni abrir un juicio público, se puede usar una dinámica de tipo "Empezar, Dejar de hacer, Seguir haciendo" enfocada específicamente en cómo se comunica el equipo, no en la calidad del código. Cada persona escribe de forma anónima en notas qué comportamiento de comunicación le gustaría que empezara, cuál le gustaría que se dejara de hacer y cuál valora que se mantenga. El facilitador agrupa las respuestas por tema, sin atribuir cada nota a quien la escribió, y el equipo habla sobre los patrones que aparecen, no sobre casos puntuales. Esto permite que el problema salga a la luz como una situación del equipo y no como una acusación directa, y le da al Arquitecto la oportunidad de escuchar el impacto de su comportamiento sin sentirse atacado frente a todos.